Operating Systems

Sheet 5 Solutions

Concurrency: Mutual Exclusion and Synchronization

Name: Yehia Said Gewily

ID: 21010705

Date: March 2026

1 Mutual Exclusion: Software Approaches (5.1)

1. Definitions

a. Mutual Exclusion:

Mutual exclusion means that only one process or thread can enter the critical section at a time. This is necessary when multiple processes share the same resource, such as a variable, file, or device, to avoid incorrect results.

b. Critical Section:

A critical section is the part of a program where shared data or shared resources are accessed. This part must not be executed by more than one process at the same time.

2. True or False: Peterson's Algorithm Cannot Be Generalized to n Processes

False.

Peterson's algorithm is originally designed for two processes, but it can be extended to more than two processes. Therefore, the statement is false.

2 Principles of Concurrency (5.2)

3. Definition of Race Condition

Race Condition:

A race condition happens when two or more processes or threads access shared data at the same time, and the final result depends on the order in which they run.

For example, if two threads both execute `counter++`, one update may be lost because this operation is not atomic.

4. Requirements for Mutual Exclusion

Any correct solution to the mutual exclusion problem should satisfy the following:

1. **Mutual Exclusion:** Only one process can be inside the critical section at a time.
2. **Progress:** If no process is in the critical section, one of the waiting processes should be allowed to enter.
3. **Bounded Waiting:** A process should not wait forever before entering the critical section.
4. **No Assumptions About Speed:** The solution must work regardless of process speed or processor speed.
5. **No Deadlock:** Processes must not get stuck waiting forever.

3 Mutual Exclusion: Hardware Support (5.3)

5. Disadvantages of the Machine-Instruction Approach

Hardware instructions such as `test-and-set` can solve mutual exclusion, but they have some disadvantages:

1. **Busy Waiting:** A waiting process keeps checking the lock in a loop, which wastes CPU time.
2. **Starvation:** Some processes may wait too long while others enter first.
3. **Priority Problems:** A high-priority process may wait for a low-priority process.
4. **Special Hardware Needed:** These methods need CPU support for special atomic instructions.
5. **Not Good for Long Critical Sections:** If the critical section takes a long time, busy waiting wastes many CPU cycles.

4 Semaphores (5.4)

6. Comparing the Two Semaphore Definitions

The two definitions are equivalent in behavior.

Both definitions of semaphores do the same job:

- `wait` decreases the semaphore or blocks the process.
- `signal` increases the semaphore or wakes up a waiting process.

The only difference is in how the value of `count` is stored internally:

- In one definition, `count` never becomes negative.
- In the other definition, `count` can become negative to show the number of waiting processes.

Conclusion: Both definitions are correct and can be used interchangeably because they produce the same external behavior.

5 General Questions

7. Concurrent `tally` Increment

Setup: Two processes each execute a function 50 times, and each time they perform `tally++`.

The statement `tally++` is actually performed in three steps:

(1) `LOAD R, tally`

(2) `INC R`

(3) `STORE R, tally`

Since these steps are not atomic, different interleavings may happen.

Part a — For Two Processes

Each process increments 50 times, so the total number of increment attempts is 100.

Maximum value of `tally` = 100

This happens if no updates are lost.

Minimum value of `tally` = 50

This happens when the operations overlap in the worst case, so two increments behave like one increment.

Final range:

$$\boxed{50 \leq \texttt{tally} \leq 100}$$

Part b — For k Processes

If there are k processes, and each process increments 50 times:

- **Maximum value:**

$$50k$$

- **Minimum value:**

$$50$$

So the final range is:

$$\boxed{50 \leq \texttt{tally} \leq 50k}$$

As the number of processes increases, the maximum value increases, but the minimum value stays the same.

8. Counterexample: Flawed Two-Process Mutual Exclusion

The proposed algorithm is:

```
void P(int id) {
    while (true) {
        blocked[id] = true;
        while (turn != id) {
            while (blocked[1-id])
                /* do nothing */;
            turn = id;
        }
        /* critical section */
        blocked[id] = false;
        /* remainder */
    }
```

```
}
```

Claim: This algorithm is incorrect because it does not guarantee proper mutual exclusion.

Reason:

The variables `blocked[]` and `turn` are not used in a correct way. A process may enter the critical section depending on `turn` without safely coordinating with the other process.

This means the algorithm can fail to protect the critical section correctly.

Conclusion: The algorithm is flawed and should not be used as a correct mutual exclusion solution.

6 Final Answers Summary

1. Definitions

- Mutual Exclusion: only one process can enter the critical section at a time.
- Critical Section: part of the code that accesses shared resources.

2. Peterson's algorithm for n processes: False.
3. Race condition: The final result depends on the order of concurrent execution.
4. Requirements of mutual exclusion:

- Mutual exclusion
- Progress
- Bounded waiting
- No speed assumptions
- No deadlock

5. Disadvantages of hardware approach:

- Busy waiting
- Starvation
- Priority problems
- Needs special hardware

- Bad for long critical sections

6. Semaphore definitions: Equivalent.

7. Range of `tally`:

$$50 \leq \texttt{tally} \leq 100$$

for 2 processes, and

$$50 \leq \texttt{tally} \leq 50k$$

for k processes.

8. Given algorithm: Incorrect because it does not guarantee proper mutual exclusion.